Chuleta: JDBC + DAO + SQL

1) Clase de conexión · 2) Qué hacer en cada método del CRUD (DAO "a pelo") · 3) Cómo simplificarlo con una clase abstracta · 4) Consultas y subconsultas útiles. Ejemplos con el esquema del Taller (clientes, vehiculos, mecanicos, reparaciones, piezas_reparacion).

1. La clase de conexión (ConexionBD)

Para qué sirve: un solo sitio donde se construye la conexión, se comprueba que el archivo existe y se activa `PRAGMA foreign_keys`. Los DAO no vuelven a escribir nada de esto.

Versión mínima (si el enunciado no pide nada más)

```
public class ConexionBD {
    private final String url;                       // "jdbc:sqlite:datos/clinica.sqlite3"
    public ConexionBD(String url) { this.url = url; }
    public Connection obtenerConexion() throws SQLException {
        return DriverManager.getConnection(url);
    }
}
```

Versión completa (la que pide el examen: validar + PRAGMA)

```
public final class ConexionBD {
    private static final String PREFIJO = "jdbc:sqlite:";
    private ConexionBD() { }

    public static Connection abrir(String url) throws SQLException {
        if (url == null || !url.startsWith(PREFIJO))
            throw new SQLException("URL SQLite no valida: " + url);
        Path ruta = Path.of(url.substring(PREFIJO.length()));
        if (!Files.isRegularFile(ruta))               // ¡SQLite CREA un archivo vacio si no existe!
            throw new SQLException("No existe la base de datos: " + ruta);

        Connection cn = DriverManager.getConnection(url);
        try (Statement st = cn.createStatement()) {
            st.execute("PRAGMA foreign_keys = ON");    // hay que hacerlo en CADA conexion
        } catch (SQLException e) {
            cn.close();                                // si algo falla, no dejar la conexion abierta
            throw e;
        }
        return cn;
    }
}
```

Opcional: comprobar que el esquema es el esperado

```
String sql = "SELECT COUNT(*) FROM sqlite_master WHERE type = 'table' "
           + "AND name IN ('clientes','vehiculos','mecanicos','reparaciones','piezas_reparacion')";
try (PreparedStatement ps = cn.prepareStatement(sql); ResultSet rs = ps.executeQuery()) {
    if (!rs.next() || rs.getInt(1) != 5) throw new SQLException("Esquema incompleto");
} catch (SQLException e) { cn.close(); throw e; }
```

Cómo se usa desde el DAO

```
private final String url;
public ClienteDao(String url) { this.url = url; }

try (Connection con = ConexionBD.abrir(url);                // 1 conexion por operacion
     PreparedStatement ps = con.prepareStatement(sql)) { ... }
// Si la clase es de instancia:  new ConexionBD(url).obtenerConexion()   /   ClienteDao extends
    ConexionBD -> obtenerConexion()
```

- La **url** la recibe el constructor del DAO (los tests pasan la de una BD temporal). No fijarla dentro del DAO.
- **Quien abre, cierra:** la conexión la cierra el *try-with-resources* del DAO, no ConexionBD.
- Ruta relativa (`datos/x.sqlite3`) se resuelve desde la carpeta donde se lanza Maven (la del pom.xml).

2. Qué hay que hacer en cada método del CRUD

Orden recomendado (cada paso se comprueba con el anterior): **insertar** → **buscarPorId** → **listar** → **actualizar** → **eliminar** → **consultas específicas**. Entre tablas, según las claves foráneas: primero las tablas padre (Cliente, Mecánico) y luego las hijas (Vehículo, Reparación, Pieza).

Método	SQL	Pasos en Java	Retorno / trampas
insertar	INSERT INTO t (cols sin id) VALUES (?,?,?)	1. Validar entrada (null, blancos). 2. prepareStatement(sql, **RETURN_GENERATED_KEYS**). 3. setXxx. 4. executeUpdate(). 5. getGeneratedKeys().	id generado (int). Si falla/inválido: 0 o excepción propia. **No** meter el id si es AUTOINCREMENT (Vehículo con matrícula sí la lleva).
buscarPorId	SELECT cols FROM t WHERE id = ?	1. id null → Optional.empty(). 2. setInt. 3. executeQuery(). 4. `if (rs.next())` mapear.	Optional.of(obj) / Optional.empty(). Nunca null.
listar	SELECT cols FROM t ORDER BY id	1. executeQuery(). 2. `while (rs.next())` mapear y añadir a ArrayList.	Lista vacía si no hay filas. Nunca null.
actualizar	UPDATE t SET a=?, b=? WHERE id=?	1. Validar (obj y id no null). 2. setXxx: **el id va el último**. 3. return executeUpdate().	Filas afectadas (1 ó 0 si no existe).
eliminar	DELETE FROM t WHERE id = ?	1. id null → 0. 2. setInt. 3. return executeUpdate().	Filas afectadas. Con FK RESTRICT falla si hay hijos → borrar hijos antes.
buscarPorX (exacta)	WHERE col = ?	Igual que listar + setString. Texto null/blanco → lista vacía.	Lista vacía si no hay.
buscarPorTexto	WHERE col LIKE ?	`ps.setString(1, "%" + txt + "%")` (los % en Java).	Lista.
buscarPorRango	WHERE col BETWEEN ? AND ?	setX(1, min); setX(2, max). Si min > max → lista vacía.	Lista.
contar / suma / media	SELECT COUNT(*) / SUM() / AVG()	executeQuery + **rs.next()** + rs.getInt(1) / getDouble(1) / getBigDecimal(1).	COALESCE(SUM(..),0) para que no dé NULL.
contarPorX	SELECT col, COUNT(*) FROM t GROUP BY col ORDER BY col	while(rs.next()) map.put(rs.getString(1), rs.getInt(2)).	LinkedHashMap (mantiene el orden).

Antes de escribir cada método: ¿qué devuelve si no hay datos? ¿y si me pasan null? ¿y si falla el SQL? (el enunciado/tests lo dicen: 0, Optional.empty(), List.of(), BigDecimal.ZERO o lanzar excepción). **NOT NULL** no impide cadenas vacías ni solo espacios: comprobar con `texto == null || texto.isBlank()` en Java.

DAO "a pelo": todo dentro del propio DAO, sin clase abstracta (empieza por aquí)

```
public class ClienteDao {
    private final String url;
    public ClienteDao(String url) { this.url = url; }

    public int insertar(Cliente c) {
        if (c == null || c.getNombre() == null || c.getNombre().isBlank()) return 0;
        String sql = "INSERT INTO clientes (nombre, email, telefono) VALUES (?,?,?)";
        try (Connection con = ConexionBD.abrir(url);
             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, c.getNombre());
            ps.setString(2, c.getEmail());
            ps.setString(3, c.getTelefono());          // null es valido: guarda NULL
            if (ps.executeUpdate() == 0) return 0;
            try (ResultSet rs = ps.getGeneratedKeys()) { return rs.next() ? rs.getInt(1) : 0; }
        } catch (SQLException e) { System.err.println("insertar: " + e.getMessage()); return 0; }
    }

    public Optional<Cliente> buscarPorId(Integer id) {
        if (id == null) return Optional.empty();
        String sql = "SELECT id_cliente, nombre, email, telefono FROM clientes WHERE id_cliente = ?";
        try (Connection con = ConexionBD.abrir(url); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next())
                    return Optional.of(new Cliente(rs.getInt("id_cliente"), rs.getString("nombre"),
                                                   rs.getString("email"), rs.getString("telefono")));
                return Optional.empty();
            }
        } catch (SQLException e) { System.err.println("buscarPorId: " + e.getMessage()); return
            Optional.empty(); }
    }

    public List<Cliente> listar() {
        String sql = "SELECT id_cliente, nombre, email, telefono FROM clientes ORDER BY id_cliente";
        List<Cliente> lista = new ArrayList<>();
```

```
        try (Connection con = ConexionBD.abrir(url);
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next())
                lista.add(new Cliente(rs.getInt("id_cliente"), rs.getString("nombre"),
                                      rs.getString("email"), rs.getString("telefono")));
        } catch (SQLException e) { System.err.println("listar: " + e.getMessage()); return List.of(); }
        return lista;
    }

    public int actualizar(Cliente c) {
        if (c == null || c.getIdCliente() == null) return 0;
        String sql = "UPDATE clientes SET nombre = ?, email = ?, telefono = ? WHERE id_cliente = ?";
        try (Connection con = ConexionBD.abrir(url); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, c.getNombre()); ps.setString(2, c.getEmail()); ps.setString(3,
                c.getTelefono());
            ps.setInt(4, c.getIdCliente());                          // el id, el ULTIMO
            return ps.executeUpdate();
        } catch (SQLException e) { System.err.println("actualizar: " + e.getMessage()); return 0; }
    }

    public int eliminar(Integer id) {
        if (id == null) return 0;
        String sql = "DELETE FROM clientes WHERE id_cliente = ?";
        try (Connection con = ConexionBD.abrir(url); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            return ps.executeUpdate();
        } catch (SQLException e) { System.err.println("eliminar: " + e.getMessage()); return 0; }
    }

    // consulta especifica: exacta, con validacion de entrada
    public List<Vehiculo> buscarPorMarca(String marca) { /* igual que listar + ps.setString(1, marca)
        */ }

    // calculo: SUM nunca devuelve null gracias a COALESCE
    public BigDecimal calcularCoste(Integer idReparacion) {
        String sql = "SELECT COALESCE(SUM(cantidad * precio_unitario), 0) FROM piezas_reparacion WHERE
            id_reparacion = ?";
        try (Connection con = ConexionBD.abrir(url); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, idReparacion);
            try (ResultSet rs = ps.executeQuery()) { rs.next(); return rs.getBigDecimal(1); }
        } catch (SQLException e) { return BigDecimal.ZERO; }
    }
}
```

Estilo clínica: en vez de devolver 0 / lista vacía, el `catch` hace `throw new DaoException("Error listando clientes", e);` (RuntimeException con la causa).

3. Simplificarlo después: de "a pelo" a AbstractDao<T>

Regla: **se mueve a la abstracta todo lo que es idéntico en todos los DAO**; se queda en el DAO lo que cambia (el SQL, los `setXxx`, la validación y cómo construir el objeto). Hazlo en pasos y ejecuta los tests entre uno y otro.

Paso	Qué se repite	Se extrae a...
1	Abrir la conexión (url, comprobar archivo, PRAGMA)	ConexionBD.abrir(url) (sección 1)
2	`new Cliente(rs.getInt(..), rs.getString(..)...)` en listar, buscarPorId, filtros	`private Cliente mapear(ResultSet rs)` en el propio DAO
3	El bucle `while(rs.next())`, el `if(rs.next())`, el executeUpdate, las claves generadas, el catch	Métodos protegidos de `AbstractDao<T>`: consultar, consultarUno, ejecutar, insertarId, escalar...
4	Lista de columnas y texto de validación repetidos	`private static final String COLUMNAS = "id_cliente,nombre,email,telefono";` y textoValido() en la abstracta

La clase abstracta (lo que se escribe UNA vez)

```
public abstract class AbstractDao<T> {
    private static final Logger LOG = Logger.getLogger(AbstractDao.class.getName());

    protected abstract Optional<T> mapRsToClass(ResultSet rs);       // cada DAO dice como convertir 1
        fila en T

    protected List<T> consultar(PreparedStatement ps) {             // SELECT -> lista
        try (ResultSet rs = ps.executeQuery()) {
            List<T> res = new ArrayList<>();
            while (rs.next()) {
```

```
                Optional<T> e = mapRsToClass(rs);
                if (e.isEmpty()) return List.of();                  // no devolver listas a medias
                res.add(e.get());
            }
            return res;
        } catch (SQLException e) { registrar("consultar", e); return List.of(); }
    }
    protected Optional<T> consultarUno(PreparedStatement ps) {      // SELECT ... WHERE id = ?
        try (ResultSet rs = ps.executeQuery()) { return rs.next() ? mapRsToClass(rs) :
            Optional.empty(); }
        catch (SQLException e) { registrar("consultarUno", e); return Optional.empty(); }
    }
    protected int ejecutar(PreparedStatement ps) {                  // UPDATE / DELETE -> filas
        try { return ps.executeUpdate(); } catch (SQLException e) { registrar("ejecutar", e); return 0;
            }
    }
    protected int insertarId(PreparedStatement ps) {                // INSERT -> id generado
        try { if (ps.executeUpdate() == 0) return 0;
              try (ResultSet rs = ps.getGeneratedKeys()) { return rs.next() ? rs.getInt(1) : 0; }
        } catch (SQLException e) { registrar("insertarId", e); return 0; }
    }
    protected int consultarEntero(PreparedStatement ps) {           // COUNT(*)
        try (ResultSet rs = ps.executeQuery()) { return rs.next() ? rs.getInt(1) : 0; }
        catch (SQLException e) { registrar("consultarEntero", e); return 0; }
    }
    protected BigDecimal consultarDecimal(PreparedStatement ps) {   // SUM / AVG
        try (ResultSet rs = ps.executeQuery()) { return rs.next() ? rs.getBigDecimal(1) :
            BigDecimal.ZERO; }
        catch (SQLException e) { registrar("consultarDecimal", e); return BigDecimal.ZERO; }
    }
    protected boolean textoValido(String t) { return t != null && !t.isBlank(); }
    protected void registrar(String op, Exception e) { LOG.log(Level.WARNING, op, e); }
}
```

El DAO después de simplificar (compara con la sección 2)

```
public class ClienteDao extends AbstractDao<Cliente> {
    private static final String COLUMNAS = "id_cliente, nombre, email, telefono";
    private final String url;
    public ClienteDao(String url) { this.url = url; }

    @Override protected Optional<Cliente> mapRsToClass(ResultSet rs) {      // la UNICA parte de mapeo
        try { return Optional.of(new Cliente(rs.getInt("id_cliente"), rs.getString("nombre"),
                                             rs.getString("email"), rs.getString("telefono"))); }
        catch (SQLException e) { registrar("mapRsToClass", e); return Optional.empty(); }
    }

    public List<Cliente> listar() {                                          // de ~10 lineas a 5
        try (Connection con = ConexionBD.abrir(url);
             PreparedStatement ps = con.prepareStatement("SELECT " + COLUMNAS + " FROM clientes ORDER
                 BY id_cliente")) {
            return consultar(ps);
        } catch (SQLException e) { registrar("listar", e); return List.of(); }
    }
    public Optional<Cliente> buscarPorId(Integer id) {
        if (id == null) return Optional.empty();
        try (Connection con = ConexionBD.abrir(url);
             PreparedStatement ps = con.prepareStatement("SELECT " + COLUMNAS + " FROM clientes WHERE
                 id_cliente = ?")) {
            ps.setInt(1, id);
            return consultarUno(ps);
        } catch (SQLException e) { registrar("buscarPorId", e); return Optional.empty(); }
    }
    public int insertar(Cliente c) {
        if (c == null || !textoValido(c.getNombre()) || !textoValido(c.getEmail())) return 0;
        try (Connection con = ConexionBD.abrir(url);
             PreparedStatement ps = con.prepareStatement("INSERT INTO clientes (nombre,email,telefono)
                 VALUES (?,?,?)",
                                                         Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, c.getNombre()); ps.setString(2, c.getEmail()); ps.setString(3,
                c.getTelefono());
            return insertarId(ps);
        } catch (SQLException e) { registrar("insertar", e); return 0; }
    }
    // actualizar / eliminar: igual, terminando en  return ejecutar(ps);
}
```

Un paso más si quieres (opcional): que la abstracta reciba el SQL y los parámetros

```
protected List<T> consultar(String sql, String url, Object... params) {
    try (Connection con = ConexionBD.abrir(url); PreparedStatement ps = con.prepareStatement(sql)) {
        for (int i = 0; i < params.length; i++) ps.setObject(i + 1, params[i]);     // setObject sirve
```

```
            para String/Integer/Double
        try (ResultSet rs = ps.executeQuery()) { ...mismo bucle... }
    } catch (SQLException e) { registrar("consultar", e); return List.of(); }
}
// Uso: consultar("SELECT ... FROM vehiculos WHERE marca = ? AND anio >= ?", url, "Seat", 2018);
```

Qué NO mover a la abstracta: el SQL concreto, la validación propia de cada entidad y el orden de los setXxx. Si la abstracta empieza a conocer nombres de columnas o de tablas, está mal diseñada. Orden de trabajo: primero que **pasen los tests a pelo**; después refactorizar y volver a pasar los tests.

4. Consultas útiles (probadas sobre la BD del taller)

Relacionar tablas

```
-- INNER JOIN: reparaciones con cliente, matricula y mecanico
SELECT r.id_reparacion, c.nombre AS cliente, v.matricula, m.nombre AS mecanico FROM reparaciones r JOIN
    vehiculos v ON v.matricula = r.matricula JOIN clientes c ON c.id_cliente = v.id_cliente JOIN
        mecanicos m ON m.id_mecanico = r.id_mecanico ORDER BY r.id_reparacion

-- LEFT JOIN + COUNT(col de la derecha): clientes con su n.o de vehiculos (0 incluido)
SELECT c.id_cliente, c.nombre, COUNT(v.matricula) AS num_vehiculos FROM clientes c LEFT JOIN vehiculos
    v ON v.id_cliente = c.id_cliente GROUP BY c.id_cliente, c.nombre

-- coste total en piezas por cliente (4 tablas, sin perder clientes)
SELECT c.nombre, COALESCE(SUM(p.cantidad * p.precio_unitario), 0) AS total FROM clientes c LEFT JOIN
    vehiculos v ON v.id_cliente = c.id_cliente LEFT JOIN reparaciones r ON r.matricula = v.matricula
        LEFT JOIN piezas_reparacion p ON p.id_reparacion = r.id_reparacion GROUP BY c.id_cliente,
            c.nombre ORDER BY total DESC

-- GROUP_CONCAT: todas las matriculas de cada cliente en una celda
SELECT c.nombre, GROUP_CONCAT(v.matricula, ', ') AS matriculas FROM clientes c JOIN vehiculos v ON
    v.id_cliente = c.id_cliente GROUP BY c.id_cliente, c.nombre
```

Subconsultas (lo que más se pregunta)

```
-- IN: clientes QUE tienen vehiculos
SELECT * FROM clientes WHERE id_cliente IN (SELECT id_cliente FROM vehiculos)

-- NOT IN: clientes SIN vehiculos (ojo: si la subconsulta devuelve algun NULL, NOT IN no devuelve nada)
SELECT * FROM clientes WHERE id_cliente NOT IN (SELECT id_cliente FROM vehiculos)

-- EXISTS (correlacionada): igual que IN, suele ser mas segura
SELECT c.* FROM clientes c WHERE EXISTS (SELECT 1 FROM vehiculos v WHERE v.id_cliente = c.id_cliente)

-- NOT EXISTS: reparaciones sin piezas (alternativa al LEFT JOIN ... IS NULL)
SELECT r.* FROM reparaciones r WHERE NOT EXISTS (SELECT 1 FROM piezas_reparacion p WHERE
    p.id_reparacion = r.id_reparacion)

-- subconsulta escalar: mecanicos que cobran mas que la media
SELECT * FROM mecanicos WHERE tarifa_hora > (SELECT AVG(tarifa_hora) FROM mecanicos)

-- el maximo CON empates (mejor que LIMIT 1 si piden 'todos los que...')
SELECT * FROM mecanicos WHERE tarifa_hora = (SELECT MAX(tarifa_hora) FROM mecanicos)

-- subconsulta en SELECT (correlacionada): n.o de reparaciones de cada vehiculo
SELECT v.matricula, (SELECT COUNT(*) FROM reparaciones r WHERE r.matricula = v.matricula) AS
    reparaciones FROM vehiculos v

-- subconsulta en FROM (tabla derivada): coste medio por reparacion
SELECT AVG(coste) FROM (SELECT id_reparacion, SUM(cantidad * precio_unitario) AS coste FROM
    piezas_reparacion GROUP BY id_reparacion)

-- subconsulta dentro de HAVING: reparaciones con mas unidades de piezas que la media
SELECT id_reparacion, SUM(cantidad) AS unidades FROM piezas_reparacion GROUP BY id_reparacion HAVING
    SUM(cantidad) > (SELECT AVG(u) FROM (SELECT SUM(cantidad) AS u FROM piezas_reparacion GROUP BY
        id_reparacion))

-- filtrar por una relacion: vehiculos con alguna reparacion PENDIENTE
SELECT * FROM vehiculos WHERE matricula IN (SELECT matricula FROM reparaciones WHERE estado =
    'PENDIENTE')

-- coste total = mano de obra + piezas (subconsulta en el SELECT)
SELECT r.id_reparacion, r.horas_trabajo * m.tarifa_hora AS mano_obra, COALESCE((SELECT SUM(p.cantidad *
    p.precio_unitario) FROM piezas_reparacion p WHERE p.id_reparacion = r.id_reparacion), 0) AS piezas
        FROM reparaciones r JOIN mecanicos m ON m.id_mecanico = r.id_mecanico
```

Cómo elegir: 'que tienen' → IN / EXISTS · 'que NO tienen' → NOT EXISTS (o LEFT JOIN ... IS NULL) · 'mayor que la media/máximo' → subconsulta escalar en el WHERE · 'por cada X, cuántos Y' → subconsulta en SELECT o LEFT JOIN + GROUP BY.

Agrupar, ordenar, CTE y ventanas

```
-- top 1: mecanico con mas reparaciones
SELECT m.nombre, COUNT(r.id_reparacion) AS n FROM mecanicos m LEFT JOIN reparaciones r ON r.id_mecanico
    = m.id_mecanico GROUP BY m.id_mecanico, m.nombre ORDER BY n DESC LIMIT 1

-- paginacion: LIMIT n OFFSET saltadas
SELECT * FROM clientes ORDER BY nombre LIMIT 3 OFFSET 3

-- por mes (fechas guardadas como texto AAAA-MM-DD)
SELECT strftime('%Y-%m', fecha) AS mes, COUNT(*) AS n FROM reparaciones GROUP BY mes ORDER BY mes

-- rango de fechas (el texto ISO se ordena como fecha)
SELECT * FROM reparaciones WHERE fecha BETWEEN '2026-01-01' AND '2026-12-31' ORDER BY fecha

-- CASE: etiquetar
SELECT id_reparacion, CASE WHEN horas_trabajo >= 4 THEN 'larga' WHEN horas_trabajo >= 2 THEN 'media'
    ELSE 'corta' END AS duracion FROM reparaciones

-- CTE: reparaciones cuyo coste en piezas supera la media
WITH coste AS (SELECT id_reparacion, SUM(cantidad * precio_unitario) AS total FROM piezas_reparacion
    GROUP BY id_reparacion) SELECT r.id_reparacion, r.descripcion, c.total FROM reparaciones r JOIN
        coste c ON c.id_reparacion = r.id_reparacion WHERE c.total > (SELECT AVG(total) FROM coste)

-- ventana: la reparacion mas reciente de cada vehiculo (ROW_NUMBER no repite; RANK admite empates)
WITH t AS (SELECT r.*, ROW_NUMBER() OVER (PARTITION BY matricula ORDER BY fecha DESC, id_reparacion
    DESC) AS rn FROM reparaciones r) SELECT id_reparacion, matricula, fecha FROM t WHERE rn = 1

-- UNION: una lista de 'contactos' con dos tablas (mismas columnas)
SELECT nombre, 'cliente' AS tipo FROM clientes UNION SELECT nombre, 'mecanico' FROM mecanicos ORDER BY
    nombre

-- existe / no existe (devuelve 0 o 1)
SELECT EXISTS (SELECT 1 FROM clientes WHERE email = 'ana@example.com')
```

Modificar con subconsultas

```
-- UPDATE: finalizar las reparaciones que ya tienen piezas
UPDATE reparaciones SET estado = 'FINALIZADA' WHERE id_reparacion IN (SELECT id_reparacion FROM
    piezas_reparacion)

-- DELETE: borrar hijos antes que el padre (FK RESTRICT): piezas de reparaciones finalizadas
DELETE FROM piezas_reparacion WHERE id_reparacion IN (SELECT id_reparacion FROM reparaciones WHERE
    estado = 'FINALIZADA')

-- INSERT ... SELECT: copiar filas de una tabla a otra con las mismas columnas
INSERT INTO copia SELECT * FROM mecanicos WHERE especialidad = 'Motor'
```

Quiero...	Escribo	Cuidado
Texto que contiene	LIKE ? con "%" + t + "%"	LIKE ignora mayúsculas en ASCII; para forzar: `COLLATE NOCASE`
Igualdad exacta	col = ?	Distingue mayúsculas
Al menos / como máximo	>= / <=	'más de' es >, 'al menos' es >=
NULL	IS NULL / IS NOT NULL	nunca = NULL
Filtrar un agregado	HAVING	WHERE no admite COUNT/SUM/AVG
Contar con LEFT JOIN	COUNT(col_de_la_derecha)	COUNT(*) cuenta 1 para los que no tienen pareja
Suma sin filas	COALESCE(SUM(x), 0)	SUM devuelve NULL